WEB SERVICES & SOA LABORATORY

LAB 8 TASK

Kubernetes, CI/CD & Monitoring

Kubernetes • GitHub Actions • Prometheus • Grafana

Lab Continuity	Build on the Lab 7 API Gateway + User/Product/Order microservices application.
Primary Goal	Deploy the existing application on Kubernetes, demonstrate a small CI/CD workflow with GitHub Actions, and monitor the running system.
Database	MongoDB Atlas remains the external managed database used in Lab 7.
Kubernetes Environment	Use a Kubernetes environment available to you (local or cloud). The task uses standard kubectl concepts.
Level	Medium • Application-based • Step-by-step

1. Lab Objective & Scope

Continue the application developed in Lab 7 and move it toward a practical DevOps workflow: Kubernetes for orchestration, basic GitHub Actions for CI, and Prometheus/Grafana for monitoring.

By the end of the task, you should demonstrate:

- Deployment of the API Gateway, User Service, Product Service and Order Service on Kubernetes.
- Use of Pods, Deployments, Services, namespaces and configuration.
- Kubernetes service discovery, scaling and self-healing.
- A GitHub Actions CI/CD workflow for build/test, versioned Docker image creation, registry delivery and Kubernetes deployment.
- Prometheus metric collection/querying and Grafana visualization.
- API traffic generation followed by observation of monitoring changes.

2. Starting Point & Prerequisites

2.1 Starting Point: Lab 7 Application

Do not rebuild the application. Use the working Lab 7 system.

- API Gateway – client-facing entry point.
- User Service.
- Product Service.
- Order Service.
- MongoDB Atlas – external managed database.
- Configuration-based service locations and Docker images from previous labs.

Expected application flow: Client/Postman → API Gateway → User/Product/Order Services → MongoDB Atlas

2.2 Prerequisites

- Working Lab 7 application and source code.
- GitHub repository containing the project.
- Docker installed and working.
- A local or cloud Kubernetes environment.
- kubectl installed and connected to the selected cluster.
- Postman or another HTTP client.
- Prometheus and Grafana available through the setup used for the lab.

Security requirement: Do not commit database credentials, tokens or passwords to GitHub.

3. Part A — Kubernetes Deployment

3.1 Verify the Lab 7 Application

- Start the Lab 7 application using your existing method.
- Verify the API Gateway is reachable.
- Test User, Product and Order operations through the gateway.
- Verify service-to-service communication and MongoDB Atlas access.
- Record the gateway address/port and image names/tags used.

Evidence: Capture a successful Postman request through the gateway and its response.

3.2 Verify the Kubernetes Environment

The exact cluster setup is environment-dependent. Once the cluster is available, use the standard workflow below.

- Check the current Kubernetes context.
- Confirm at least one node is available.
- Create a namespace for Lab 8.
- Use the Lab 8 namespace for application resources.

```
kubectl config current-context
kubectl get nodes
kubectl create namespace lab8
```

Checkpoint: Show the active context and at least one available node.

3.3 Create Kubernetes Manifests

Create a k8s/ directory in the Lab 7 repository.

```
k8s/
  gateway-deployment.yaml   gateway-service.yaml
  user-deployment.yaml      user-service.yaml
  product-deployment.yaml   product-service.yaml
  order-deployment.yaml     order-service.yaml
  configmap.yaml
```

- Deployment + Service for API Gateway.
- Deployment + Service for User Service.

- Deployment + Service for Product Service.
- Deployment + Service for Order Service.
- ConfigMap for appropriate non-sensitive environment configuration.
- Correct images, ports, labels and selectors.

Database: MongoDB Atlas remains external to the Kubernetes cluster, as in Lab 7.

3.3.1 Health & Readiness Probes

Add liveness and readiness probes to the appropriate Kubernetes Deployment(s). Use the existing application health endpoint where available.

Liveness checks whether the application is alive; readiness checks whether the Pod is ready to receive traffic. Readiness is especially important during startup and rolling updates.

Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
readinessProbe:
  httpGet:
    path: /health
    port: 8080
```

Evidence: Show the probe configuration and verify the Pod reports Ready.

3.4 Configuration & Service Discovery

- Use ConfigMap for non-sensitive values such as URLs, ports or feature configuration.
- Use Secret for sensitive configuration when required.
- Never use individual Pod IPs as permanent addresses.
- Use Kubernetes Service names for internal communication.
- Keep environment-specific configuration outside images where possible.

Example: Internal traffic can use a stable name such as http://user-service:<port>.

3.5 Deploy & Verify

- Apply the manifests.
- Check Deployments, Pods and Services.
- Use logs/describe output when a workload does not start.

```
kubectl apply -f k8s/ -n lab8
kubectl get deployments -n lab8
kubectl get pods -n lab8
kubectl get services -n lab8
```

Checkpoint: Required Deployments should reach the expected state and Pods should become ready.

For a simple local setup, NodePort may be used. Cloud environments may use LoadBalancer or Ingress.

Example: Gateway Service → NodePort → http://<Node-IP>:<NodePort>. Keep the API Gateway as the client-facing entry point.

3.6 Expose & Test the Gateway

- Expose the gateway using the appropriate external access mechanism for your Kubernetes environment.
- Obtain its address/port.
- Send requests from Postman to the gateway.
- Verify User, Product and Order operations.
- Confirm internal routing uses Kubernetes Services.

Evidence: Capture the gateway Service and at least two successful Postman requests.

3.7 Scaling

- Choose the User Service Deployment.
- Scale it from one replica to three.
- Check the Pods.
- Verify the Service remains the stable access point.

```
kubectl scale deployment user-service --replicas=3 -n lab8
kubectl get pods -n lab8
```

3.8 Self-Healing

- List the Pods.
- Identify a User Service Pod managed by the Deployment.
- Delete that Pod.
- Check the Pods again.
- Observe Kubernetes creating a replacement to restore the desired state.

```
kubectl get pods -n lab8
kubectl delete pod <pod-name> -n lab8
kubectl get pods -n lab8
```

Evidence: Show the deleted Pod and the replacement Pod becoming ready/running.

3.9 Troubleshooting

Use standard Kubernetes inspection commands.

```
kubectl describe pod <pod-name> -n lab8
kubectl logs <pod-name> -n lab8
kubectl get endpoints -n lab8
```

- For Pod failures, inspect describe output and logs.
- For unreachable Services, check selectors, ports, endpoints and readiness.
- For configuration issues, check the supplied environment/configuration.

4. Part B — GitHub Actions CI/CD

Continuous Integration (CI) automatically builds and tests code changes. Continuous Delivery (CD) delivers a validated artifact toward the deployment environment. Continuous Deployment automatically deploys a validated version.

For this lab, use a lightweight flow: Git Push → Build/Test → Docker Image → Container Registry → Kubernetes Deployment → Verification.

4.1 Understand the CI/CD Workflow

Create a workflow under .github/workflows/ that runs when code changes.

```
.github/workflows/ci.yml
```

- Trigger on push and/or pull_request.
- Use an Ubuntu runner.
- Checkout the repository.
- Install dependencies using the project's actual technology.
- Run tests.
- Build the Docker image.

Basic structure:

```
name: CI
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
      - run: docker build -t app:${{ github.sha }} .
```

Adaptation: Replace npm commands with commands used by the actual project. If the repository contains multiple services, ensure the required services are checked and their Docker images can be built.

4.2 Container Registry & Versioned Images

A container registry stores versioned Docker images so Kubernetes can pull the exact application version produced by the CI/CD pipeline.

Use Docker Hub, GitHub Container Registry (GHCR), or another available registry. Prefer a traceable tag such as the Git commit SHA.

Example:
docker build -t <registry>/<username>/user-service:${GITHUB_SHA} .
docker push <registry>/<username>/user-service:${GITHUB_SHA}

Evidence: Show the versioned image in the selected container registry.

4.3 Verify GitHub Actions CI/CD

4.2 Verify GitHub Actions

- Push the workflow to GitHub.
- Open Actions and confirm the workflow runs.
- Verify checkout, dependency installation, tests and Docker build.
- Make a small valid change and push again to demonstrate a new CI run.

Evidence: Capture a successful GitHub Actions run showing completed CI steps.

5. Part C — Monitoring with Prometheus & Grafana

Monitoring flow: Application /metrics → Prometheus scrape → Target UP → PromQL → Grafana.

If the application exposes /metrics, configure Prometheus to scrape the appropriate endpoint. The exact configuration depends on the selected Kubernetes environment.

Example:

```
scrape_configs:
 - job_name: "gateway"
   static_configs:
    - targets: ["gateway-service:8080"]
```

Evidence: Show the Prometheus Targets page with the target in UP state and run at least one useful PromQL query.

5.1 Prometheus Monitoring

- Ensure Prometheus is running and can reach configured targets.
- Identify the targets being scraped.
- Check whether targets report UP.
- Use the query interface to inspect useful metrics.
- Use metrics that answer questions about availability, traffic, errors or latency.

```
up
rate(http_requests_total[5m])
```

Environment note: Exact target configuration depends on the Kubernetes environment and how application metrics are exposed. Use the metrics/exporter setup available for the lab.

5.2.1 Application vs Infrastructure Metrics

Application metrics describe requests, errors, latency and availability. Infrastructure/Kubernetes metrics describe Pods, containers, nodes and resource usage.

For this lab, prioritize application request/error/latency metrics together with basic Kubernetes workload health.

5.2 Create a Grafana Dashboard

- Connect Grafana to Prometheus.
- Create a Lab 8 dashboard.
- Add availability where supported.
- Add traffic/request-rate where supported.
- Add error-related information where supported.
- Add latency/duration where supported.
- Keep the dashboard focused and ensure every panel answers a monitoring question.

Suggested monitoring questions: Are targets reachable? How much traffic is arriving? Are failures increasing? How long are requests taking?

5.3 Generate Traffic & Observe

- Send several normal API requests through the Kubernetes gateway using Postman.
- Inspect corresponding metrics in Prometheus.

- Open Grafana and observe the dashboard.
- Generate additional requests and compare the dashboard before/after.
- If supported, perform a safe controlled invalid-request/failure experiment and observe relevant metrics.

Evidence: Capture one Prometheus query/result and one Grafana dashboard view after traffic generation.

6. Part D — End-to-End Integration

6.1 Complete Workflow

```
Lab 7 Application
   ↓
Kubernetes Deployment
   ↓
Verify / Health Probes / Scale / Self-heal
   ↓
GitHub Actions CI/CD
   ↓
Prometheus
   ↓
Grafana
```

6.2 Application Architecture

```
Client / Postman
      ↓
API Gateway Service
      ↓
User / Product / Order Services
      ↓
MongoDB Atlas
```

Also show the versioned Docker image in the container registry and, where implemented, the Kubernetes rolling update to that image.

6.3 Final Verification

- Show the application running on Kubernetes.
- Show the gateway as the client-facing entry point.
- Show internal communication through Kubernetes Services.
- Show scaling and self-healing.
- Show a successful GitHub Actions CI/CD run.
- Show Prometheus targets/queries.
- Show the Grafana dashboard responding to generated traffic.

7. Submission & Documentation

7.1 Required Evidence / Deliverables

No.	Evidence	What to show
1	Lab 7 baseline	Successful gateway

		request/response.
2	Kubernetes environment	Current context + node(s).
3	Manifests	Gateway, service and configuration YAML files.
4	Deployment	Pods, Deployments and Services in lab8.
5	Gateway test	Successful Postman request through Kubernetes gateway.
6	Scaling	User Service scaled to 3 replicas.
7	Self-healing	Deleted Pod followed by replacement.
8	Troubleshooting	Useful describe/logs/endpoints output.
9	GitHub Actions	Successful basic CI workflow run.
10	Prometheus	Targets/UP state and at least one query.
11	Grafana	Monitoring dashboard.
12	Traffic	Monitoring after API traffic.
13	Architecture	Final architecture diagram/screenshot.

Container registry/image naming and tagging approach.

CI/CD workflow stages and Kubernetes image update/rollout procedure.

Health/readiness probe configuration and expected behavior.

7.2 README Requirements

- Application overview and Lab 7 starting point.
- Prerequisites and selected Kubernetes environment.
- How to access the cluster/context.
- How to apply manifests and verify resources.
- How to access/test the gateway.
- Scaling and self-healing commands.
- GitHub Actions workflow description.
- Prometheus target/metric information.
- Grafana dashboard and monitored metrics.
- Configuration names without exposing secrets.
- Troubleshooting notes and evidence references.

7.3 Good Practices

- Keep environment-specific configuration outside images where possible.
- Use meaningful Kubernetes names and consistent labels.
- Never commit credentials, tokens or database passwords.
- Use versioned Docker image tags.
- Keep workflows small, readable and deterministic.
- Use Service names instead of Pod IPs.
- Expose only the gateway externally unless required.

- Monitor metrics that answer real operational questions.
- Keep code, manifests, architecture and README consistent.

7.4 Submission Checklist

☐ Lab 7 application verified.
☐ Kubernetes context/node verified.
☐ lab8 namespace created and used.
☐ Gateway/User/Product/Order Deployments and Services created.
☐ ConfigMap/Secret used appropriately.
☐ Application deployed and verified.
☐ Gateway tested from Postman.
☐ User Service scaled to 3 replicas.
☐ Self-healing demonstrated.
☐ GitHub Actions CI/CD workflow passed.
☐ Prometheus targets/metrics verified.
☐ Grafana dashboard created.
☐ API traffic generated and observed.
☐ Versioned Docker image pushed to the selected container registry.
☐ Kubernetes rollout to the new image verified.
☐ README updated.
☐ All required screenshots/evidence collected.